DATA20046: Neural Networks and Deep Learning

Programming assignments appended to the Exam of 9th Dec 2025

Instructions:

- Completing the course requires both an exam and a set of exercises. If you did not complete the exercise problems during the Spring 2025 course, you need to solve this alternative set of problems. This is obligatory for completing the course – passing the exam alone is not enough.
- The problems intentionally do not specify many of the choices (e.g. which optimizer to use, how many samples to use for training, ...), but rather the idea is that you need to recognize these aspects and make educated choices based on the course material. For each such element, you should explain the choice you made and give a brief justification.
- The grading focuses on correctness of the concepts and terminology, not on the numerical results. The presentation matters as the solutions are graded based on how we understand the solutions.
- The solutions should be returned by email to aapo.hyvarinen@helsinki.fi and arto.klami@helsinki.fi. Your submission should include a single pdf document gathering together the answers for all tasks, providing the requested plots and explanations and written so that it is self-contained and easy to follow. In addition, return a zip package including all of the code (can be a Jupyter notebook).
- We recommend returning the solutions **before** the exam as the problems support learning the course material, but the final deadline is 6 weeks from the exam date. (However, if your study right ends around the end of the year, you have to return the exercises before your study right ends.)
- Use of generative AI (large language models) in creating any part of the solutions is forbidden, but you may use the tools for learning purposes and for minor textual editing. Include a description of which tools were used and how, or state clearly that no AI tools were used.

## Task 0: Prepare environment

The problems are to be completed using the `PyTorch` library. For all experiments you will be using the same data collection, the classic MNIST data set of handwritten digits. If you are not familiar with `PyTorch`, start by reading through tutorial material (see Moodle) and make sure you can download the MNIST data and e.g. plot individual samples as images or partition the data into training and validation subsets. You can also consider other data sources if you want, but need to explain them in sufficient detail for us.

## Task 1: Supervised learning basics

Train *supervised classifiers* for the digits, by following the steps below. It is important to report the intermediate steps, designed to support robust training of the model.

1. Create *two* different neural network architectures that take the images as inputs and output a ten-component vector with probabilities for the different classes. Explain the network architectures, the loss used for training, and justify your choices.
2. Split the data into three sets of *training*, *validation* and *test* samples.
3. Show that you can optimize the models well on the *training* data. Plot the objective as a function of iterations or epochs, and make sure you optimize it well. Explain how you chose the hyperparameters of the learning algorithm. Show also what would happen with poor choices.
4. Learn a good model for the data and report the *test* accuracy for the two model candidates you have. Explain how you found the good model (e.g. choice of hyperparameters) and discuss the results briefly.

## Task 2: Double descent

Read `https://arxiv.org/abs/2303.14151` that discusses the double descent phenomenon (and see the corresponding material in the lectures). Then try to *replicate the key characteristics of the double descent phenomenon* on the MNIST data, using a suitable model architecture for the investigation.

Explain how you studied this, draw appropriate visual illustrations and give the numerical results, and provide conclusions. Note that the grading is based on how you approached the problem (e.g. whether you varied the right properties of the models and made the right conclusions based on your results), with little to no attention paid on whether you observe the same properties as in the paper. It may be difficult to clearly replicate the findings since the magnitude of effect strongly depends on the data and the model.

## Task 3: Transfer learning

Try *transfer learning*.

1. Train a model for classification of MNIST digits as in Task 1 (you can probably start with one of the models you already trained).

2. Transfer the model for a new task that differs from the original. You can either change the samples for the new task (e.g. transpose, invert or rotate the digits) while keeping the classes the same, or you can change the classes (e.g. train only on digits 0-6 and consider a new task with digits 7-9 or perhaps 0-9). Remember that you should assume only a small number of training observations to be available for the new task.

3. Select a transfer learning method that is appropriate for transferring the original model for the new task. Explain the method briefly. Note that it is not important to use a particularly fancy solution, but you should choose a method that is suitable for the setup you created.

4. Evaluate the method. Run an experiment where you compare three solutions (a) the original model evaluated in the new task, (b) the same model trained only on the small set of training samples from the new task, and (c) the solution that transfer knowledge from the original model. Explain your experimental setup and report the results with appropriate plots and numerical results, with discussion.

## Task 4: Autoencoders

Train a dimension-reducing autoencoder which has only two units in the "main" hidden layer that performs the encoding, i.e. the output of $\mathbf{g}$ in the notation of the slides has a dimension of $m = 2$. Try two different architectures for the encoding and decoding networks, using the same depth for encoding and decoding layers. There are many details to be fixed here, but they are left to you; please justify your choices in the report, if any justification is possible. **Report** the resulting two 2D visualizations of the data, so that you plot 500 randomly selected MNIST digits on the 2D plane.

## Task 5: Generative models

Train a GAN. Decide the architecture and other details by yourself, but don't try anything too big and ambitious! Remember that the lecture slides pointed out that GAN training is quite difficult: if you cannot make it work within a reasonable amount of time, don't waste too much time on it; an honest and correct attempt will give you almost the maximum points. You are allowed to use code downloaded from the Internet if you prefer, and you can also use another variant of GAN apart from the original one. **Report** ten generated data points (supposed to be similar to MNIST digits). Discuss whether the generation appears successful or not, and if not, what might be the reason.